



Internet Programming

Student Name: Parastoo Shojaei Sarcheshmeh

Student ID: 25828319

Tutor Name: Ahmed Basil

Module: CSYM019 Internet Programming

Assessment: AS1 Dynamic Web Application Project

Project Title: EventHub — Events Management Web Application

Contents

1	Introduction	5
2	Project Requirements and Objectives	6
2.1	Functional Requirements	6
2.2	Non-Functional Requirements	7
3	System Design and Architecture	7
3.1	Overall System Architecture	7
3.2	Public User Workflow	8
3.3	Admin Workflow	9
4	Database Design	10
4.1	Database Overview	10
4.2	Tables	10
4.3	Entity Relationship Diagram	11
4.4	Database Justification	12
5	Implementation	12
5.1	Admin Login	12
5.2	Add Event Functionality	14
5.3	Edit Event Functionality	15
5.4	Delete Event Functionality	15
5.5	Public Event Listing	16
5.6	Search and Filter Functionality	17
6	Use of Required Technologies	18
6.1	HTML	18
6.2	CSS	18
6.3	JavaScript	19
6.4	JSON	19
6.5	AJAX	19
6.6	PHP	20
6.7	MySQL	20
7	Security and Validation	21
7.1	Input Validation	22

7.2	Data Sanitisation	22
7.3	SQL Injection Prevention	23
7.4	Admin Protection.....	23
8	Code Quality and Maintainability	24
8.1	File Organisation	24
8.2	Reusable Code.....	24
8.3	Naming and Comments	25
8.4	Semantic HTML.....	25
8.5	GitHub Commit History.....	25
9	Testing and Evaluation	26
9.1	Testing Strategy	26
9.2	Test Cases	26
9.3	PHPUnit Test Results	27
9.4	Evaluation Against Requirements.....	28
10	Challenges and Reflections.....	28
10.1	Docker Configuration.....	28
10.2	AJAX Implementation.....	29
10.3	Form Validation	29
10.4	Design Decisions.....	29
10.5	Limitations and Future Improvements	29
11	Innovation and Additional Features	30
11.1	Location Filter in the Sidebar	30
11.2	Event Counts in the Sidebar.....	30
11.3	Coming Soon Badge	30
11.4	Admin Dashboard with Statistics	30
11.5	Event Time Field	30
11.6	Hero Section.....	30
11.7	Human-Readable Date Format	31
11.8	Back to Top Button.....	31
11.9	PHPUnit Automated Testing.....	31
11.10	Colour-Coded Category Badges	31

11.11	Responsive Design	31
12	Deployment and Setup Instructions.....	31
12.1	Required Software	31
12.2	Installation Steps	32
12.3	Admin Login Details	32
12.4	Database Credentials	32
12.5	Stopping the Application	33
13	Conclusion	33
14	AI Usage Statement.....	34
15	References	34
16	Appendices	35
16.1	Appendix A: Project File Structure	35
16.2	Appendix B: Database Sample Data	35
16.3	Appendix C: PHPUnit Test File Location	35

1 Introduction

EventHub is a community events management web application. The project simulates a real world scenario where a software agency has been commissioned to build a web portal for a community organization that hosts regular events and workshops.

The organization needed a system that would allow non-technical staff to manage event content easily, while also providing an engaging experience for members of the public who want to browse and find upcoming events. Before this system existed, managing events would have required manually updating static web pages or using a spreadsheet, which is inefficient and difficult to maintain.

The application has two main sides. The first is a public-facing events listing page where any visitor can browse upcoming events, search by keyword, and filter results by category or location. The second is a secure admin panel where authorized staff can log in and manage all events, including adding new ones, editing existing ones, and deleting events that are no longer relevant.

The main features of the application are:

- A public events listing page that loads events dynamically from a MySQL database using PHP
- A keyword search and category and location filter that update results instantly using JavaScript, AJAX, and JSON without reloading the page
- A secure admin login system using PHP sessions and bcrypt password hashing
- An admin dashboard showing event statistics broken down by category
- Full event management including add, edit and delete functionality with form validation on both the client and server side
- A Coming Soon badge that automatically highlights events happening within the next seven days
- PHPUnit automated tests to verify core application logic

The technologies used in this project are HTML5, CSS3, JavaScript, PHP, MySQL, AJAX, and JSON, all running inside a Docker environment.

2 Project Requirements and Objectives

This section outlines the key requirements of the project and explains how the EventHub application responds to each one.

2.1 Functional Requirements

The following functional requirements from the assignment brief and implemented in the application:

- Admin users can log in to a secure admin panel using a username and password
- Admin users can add new events using a form with title, date, time, location, category and description fields
- Admin users can edit existing events and save the updated details to the database
- Admin users can delete events with a confirmation prompt to prevent accidental deletion
- Public users can view a full list of upcoming events loaded from the MySQL database
- Public users can search events by keyword and results update instantly without reloading the page
- Public users can filter events by category using a dropdown or by clicking a category in the sidebar
- Public users can filter events by location by clicking a location in the sidebar
- The server returns filtered event data as JSON in response to AJAX requests
- All event data is stored in and retrieved from a MySQL database
- Forms include validation on both the client side and the server side
- Error messages are displayed when required fields are missing or input is invalid
- Each event displays a title, date, time, location, category badge and description
- The admin dashboard displays total event counts broken down by all six categories
- Events happening within the next seven days automatically display a Coming Soon badge
- The number of events in each category and location is shown in the sidebar
- Clicking a category or location in the sidebar instantly filters the event list via AJAX
- A back to top button appears when the user scrolls down the page
- A hero section is displayed at the top of the public page with a welcome message
- Date is displayed in a human-readable format such as June 12, 2026 rather than 2026-06-12
- Event time is stored and displayed alongside the date on each event card
- PHPUnit automated tests verify core application logic including input validation, password hashing and XSS protection
- 24 sample events across six categories are included in the database for demonstration purposes

2.2 Non-Functional Requirements

The following non-functional requirements were considered throughout development:

- The layout is responsive and adjusts for different screen sizes using CSS Grid and media queries
- The interface is clean, modern and easy to use for non-technical staff
- The code is organised into separate files for HTML, CSS, JavaScript, and PHP to keep it maintainable
- All database queries use PDO prepared statements to prevent SQL injection
- All output is escaped using `htmlspecialchars` to prevent cross-site scripting
- Navigation is clear and consistent across all pages
- Error handling is in place on both the front end and back end
- The application was tested in Chrome and Firefox
- Version control was maintained throughout development using GitHub Classroom with regular commits
- Docker is used to ensure a consistent development environment across different machines
- The database is set up automatically using an `init.sql` file when Docker starts

3 System Design and Architecture

This section explains how the different parts of the application fit together and how data flows between the front end, back end, and database.

3.1 Overall System Architecture

EventHub follows a three-tier architecture. The front end is what sees in the browser, built with HTML, CSS and JavaScript. The back end is handled by PHP running on an Apache web server inside Docker. The database is MySQL, also running in a Docker container.

When a user opens the public site, the browser sends a request to Apache, which runs a PHP script. The PHP script connects to MySQL using PDO, retrieves the events, and sends them back as HTML. When the user searches or filters, JavaScript sends an AJAX request to a PHP script, which queries MySQL and returns the results as JSON. JavaScript then updates the page without a full reload.

The admin panel works in a similar way but requires a valid PHP session to access. Without logging in, any attempt to reach the admin pages redirects the user back to the login page.

The Docker setup includes three containers running together using Docker Compose. The first is the PHP and Apache container which serves the website. The second is the MySQL container which stores all the data. The third is Adminer which provides a visual interface for viewing and managing the database during development.

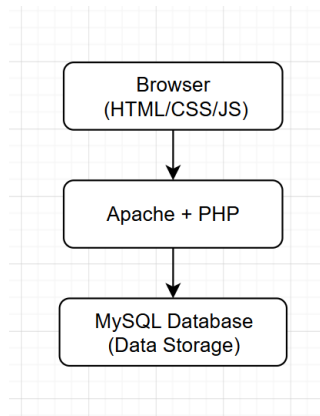


Figure 1: System Architecture

3.2 Public User Workflow

This is what happens when a visitor uses the public site:

1. The user opens the EventHub homepage in their browser
2. The browser sends a GET request to Apache
3. PHP connects to MySQL and retrieves all upcoming events ordered by date
4. The events are embedded into the HTML page and sent back to the browser
5. The user types a keyword into the search box or clicks a category or location in the sidebar
6. JavaScript detects the input and sends an AJAX GET request to `get_events.php`
7. `get_events.php` receives the search, category and location parameters
8. PHP builds a dynamic SQL query using prepared statements and queries MySQL
9. The results are encoded as JSON and returned to the browser
10. JavaScript receives the JSON response and updates the event list on the page without reloading

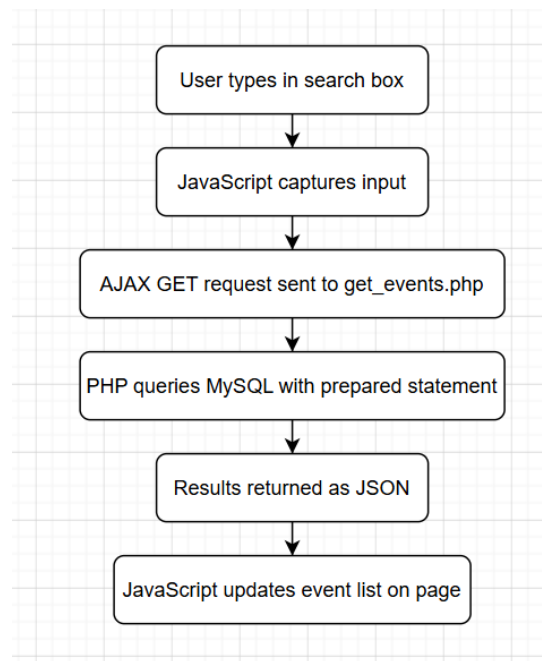


Figure 2: Public Search Workflow

3.3 Admin Workflow

This is what happens when an admin manages events:

1. The admin navigates to the login page
2. They enter their username and password
3. PHP retrieves the stored password hash from MySQL and uses `password_verify` to check it
4. If the login is successful, PHP creates a session and redirects to the admin dashboard
5. The admin can see a dashboard with event counts broken down by category
6. The admin fills in the add event form and submits it
7. JavaScript validates the form on the client side before sending it
8. The form data is sent via AJAX to `add_event.php`
9. PHP validates the data again on the server side
10. PHP uses a prepared statement to insert the new event into MySQL
11. A success message is returned as JSON and displayed on the page
12. The event list refreshes to show the new event
13. For editing, the admin clicks Edit on an event, fills in the pre-loaded form, and submits

14. For deleting, the admin clicks Delete, confirms the prompt, and the event is removed from MySQL

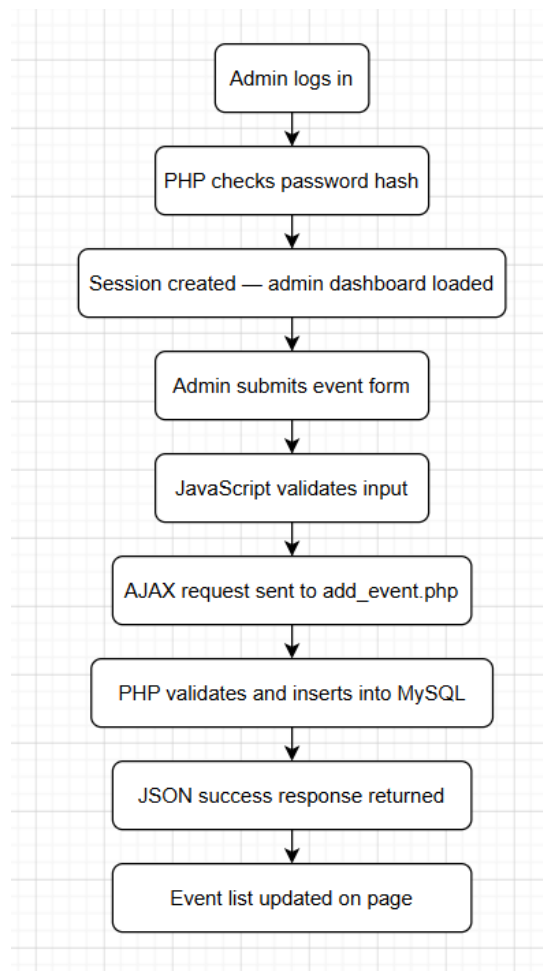


Figure 3: Admin Event Management Workflow

4 Database Design

4.1 Database Overview

[1] The EventHub application uses a MySQL database called `Internet_programming`. The database stores all event information that is displayed on the public site and managed through the admin panel. It also stores the admin login credentials. The database is created automatically when Docker starts, using an `init.sql` file that sets up the tables and inserts sample data.

4.2 Tables

The database has two tables.

Table 1: events

Stores all event information displayed on the public site

Field	Type	Description
id	INT AUTO_INCREMENT PRIMARY KEY	Unique identifier for each event
title	VARCHAR(255) NOT NULL	The name of the event
date	DATE NOT NULL	The date the event takes place
time	TIME DEFAULT '09:00:00'	The time the event starts
location	VARCHAR(255) NOT NULL	Where the event is held
category	VARCHAR(100) NOT NULL	Category such as IoT, AI or Cybersecurity
description	TEXT NOT NULL	Full description of the event
created_at	TIMESTAMP DEFAULT NOW()	When the event was added to the database

Table 2: admins

This table stores the admin login details.

Field	Type	Description
id	INT AUTO_INCREMENT PRIMARY KEY	Unique identifier for each admin
username	VARCHAR(100) NOT NULL	The admin username
password_hash	VARCHAR(255) NOT NULL	The bcrypt hashed password
created_at	TIMESTAMP DEFAULT NOW()	When the account was created

4.3 Entity Relationship Diagram

The application uses two tables that are independent of each other. The events table stores all public content and the admins table stores login credentials. There is no foreign key relationship between them because admins manage events through the application rather than through a direct database link.

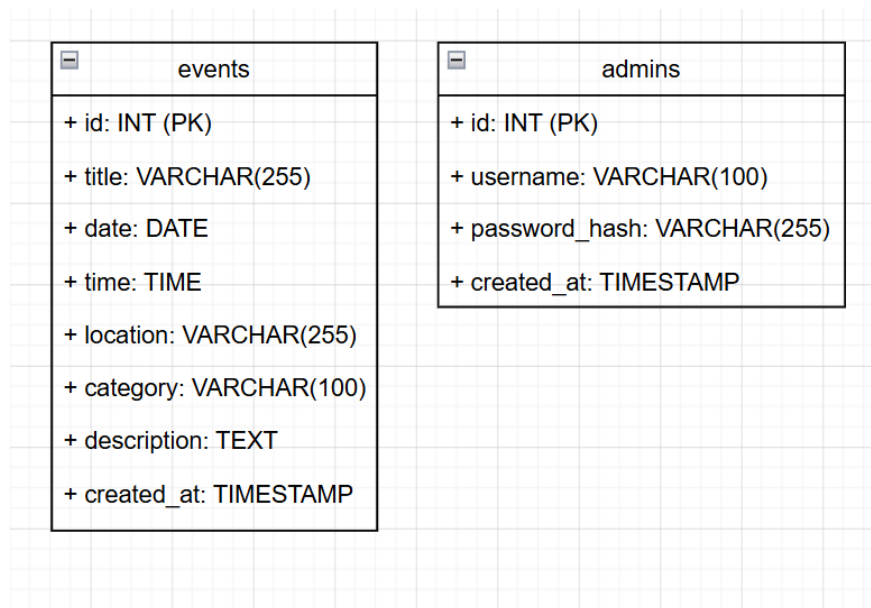


Figure 4: ERD diagram

4.4 Database Justification

I used two separate tables to keep event data and admin credentials completely separate.

Each table has a primary key using `AUTO_INCREMENT` so every record has a unique ID. This ID is used when editing or deleting a specific event — the PHP script receives the ID and uses it in a prepared statement to target the correct record.

The events table uses a `DATE` field for the date and a `TIME` field for the time separately. This makes it easier to sort events by date and to check whether an event is coming soon by comparing the date against today's date using PHP.

The category field is stored as a `VARCHAR` rather than a foreign key to a separate categories table. This keeps the database simple and is easier to manage for a small community events application.

The `init.sql` file creates both tables automatically and inserts 24 sample events across six categories so the marker can see the application working straight away without having to add any data manually.

5 Implementation

This section explains how each feature of the EventHub application was built.

5.1 Admin Login

[9] The admin login system uses PHP sessions and `bcrypt` password hashing. There is a single admin account stored in the admins table in MySQL. When the admin submits the login form, PHP retrieves

the stored password hash from the database and uses the built-in password_verify function to check whether the entered password matches. If it does, a session is created and the admin is redirected to the dashboard. If not, an error message is displayed.

Every admin page starts with a session check at the top of the PHP file. If no valid session exists, the user is immediately redirected back to the login page. This means the admin panel cannot be accessed without logging in first. A logout page destroys the session and redirects to the login page.

The login page is styled to match the rest of the EventHub design with a centred login box, username and password fields, and a submit button.

Admin Login:

```
GET username and password from POST request
QUERY database for admin WHERE username matches
IF admin found THEN
    IF password_verify(entered password, stored hash) THEN
        START session
        SET session variable 'admin' = username
        REDIRECT to admin dashboard
    ELSE
        SHOW error message "Invalid credentials"
    END IF
ELSE
    SHOW error message "Invalid credentials"
END IF
```

The screenshot shows the Admin Login page. At the top, a dark header bar contains the text "Admin Login" on the left and "Community Events Portal" on the right. The main body of the page is a light gray color. In the center, there is a white rounded rectangle containing the login form. The form has a title "Sign In" with a red vertical bar to its left. Below the title are two labels, "USERNAME:" and "PASSWORD:", each followed by a white input field. Under the input fields is a dark gray button with the text "Login". Below the button is a link that says "Back to Public Site". At the bottom of the page, a dark footer bar contains the text "© 2020 Community Events Portal".

Figure 5: Admin Login Page

5.2 Add Event Functionality

The add event form is displayed at the top of the admin dashboard. It includes fields for title, date, time, location, category and description. When the admin submits the form, JavaScript first validates the input on the client side checking that all fields are filled in and that the date is not in the past. If validation passes, the form data is sent via AJAX to `add_event.php`.

On the server side, PHP validates the input again to ensure nothing was bypassed. It then uses a PDO prepared statement to insert the new event into the MySQL database. A JSON response is returned to the browser with either a success or error message. If successful, the page reloads automatically to show the new event in the list.

Add Event:

```
CLIENT SIDE (JavaScript):
    ON form submit
      GET all field values
      IF any field is empty THEN show error, STOP
      IF date is in the past THEN show error, STOP
      SEND form data via AJAX POST to add_event.php

SERVER SIDE (PHP):
    GET title, date, time, location, category, description from POST
    IF any field is empty THEN return JSON error
    PREPARE INSERT statement with PDO
    EXECUTE with sanitised values
    RETURN JSON success response
```

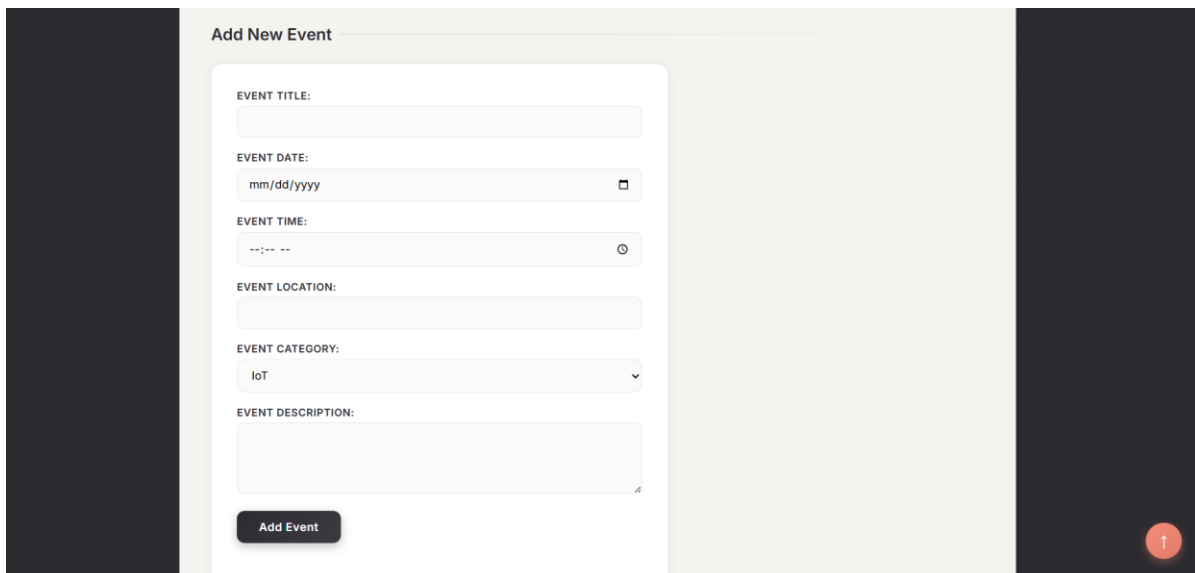


Figure 6: Add Event Form

5.3 Edit Event Functionality

When the admin clicks Edit on an event, they are taken to `edit_event.php` with the event ID passed in the URL. PHP uses the ID to retrieve the event from the database and pre-fills the form with the existing values. The admin can then change any field and submit the form.

On submission, PHP validates the updated data and uses a prepared statement to update the record in the database using the event ID. Once saved, the admin is redirected back to the dashboard.

Edit Event:

```
GET event ID from URL parameter
QUERY database SELECT * FROM events WHERE id = :id
PRE-FILL form with existing event data

ON form submit:
    GET updated values from POST
    IF any field is empty THEN show error
    PREPARE UPDATE statement
    SET title, date, time, location, category, description WHERE
id = :id
    EXECUTE with sanitised values
    REDIRECT to admin dashboard
```

The screenshot shows the 'EventHub Admin' interface. On the left is a dark sidebar with links: 'Public Site', 'Dashboard', and 'Logout'. The main content area is titled 'Edit Event' and contains a form with the following fields: 'Event Title' (IoT Smart Home Basics), 'Event Date' (06/12/2026), 'Event Time' (09:00 AM), 'Event Location' (Northampton), 'Event Category' (IoT), and 'Event Description' (Introduction to sensors, microcontrollers, and smart home systems.). There are 'Save Changes' and 'Cancel' buttons at the bottom right of the form. The footer of the page reads '© 2026 EventHub'.

Figure 7: Edit Event Form

5.4 Delete Event Functionality

Each event card in the admin panel has a Delete button. When clicked, a JavaScript confirmation prompt appears asking the admin to confirm the deletion. If confirmed, an AJAX POST request is sent to

delete_event.php with the event ID. PHP receives the ID and uses a prepared statement to delete the matching record from the database. A JSON success response is returned and the page reloads to reflect the change.

Delete Event:

```
ON delete button click
  GET event ID from button data attribute
  SHOW confirmation prompt
  IF admin confirms THEN
    SEND AJAX POST to delete_event.php with event ID
    SERVER: DELETE FROM events WHERE id = :id
    IF success THEN reload page
    ELSE show error message
  END IF
```

5.5 Public Event Listing

The public site loads all upcoming events from the MySQL database when the page opens. PHP connects to MySQL using PDO and runs a SELECT query to retrieve all events ordered by date. The results are looped through in PHP and each event is embedded directly into the HTML as an event card showing the title, date, time, location, category badge and description.

Events happening within the next seven days automatically display a Coming Soon badge. This is calculated in PHP by comparing the event date against today's date using the DateTime class.

The sidebar shows all categories and locations with a count of how many events are in each one. Clicking any category or location filters the event list instantly via AJAX.

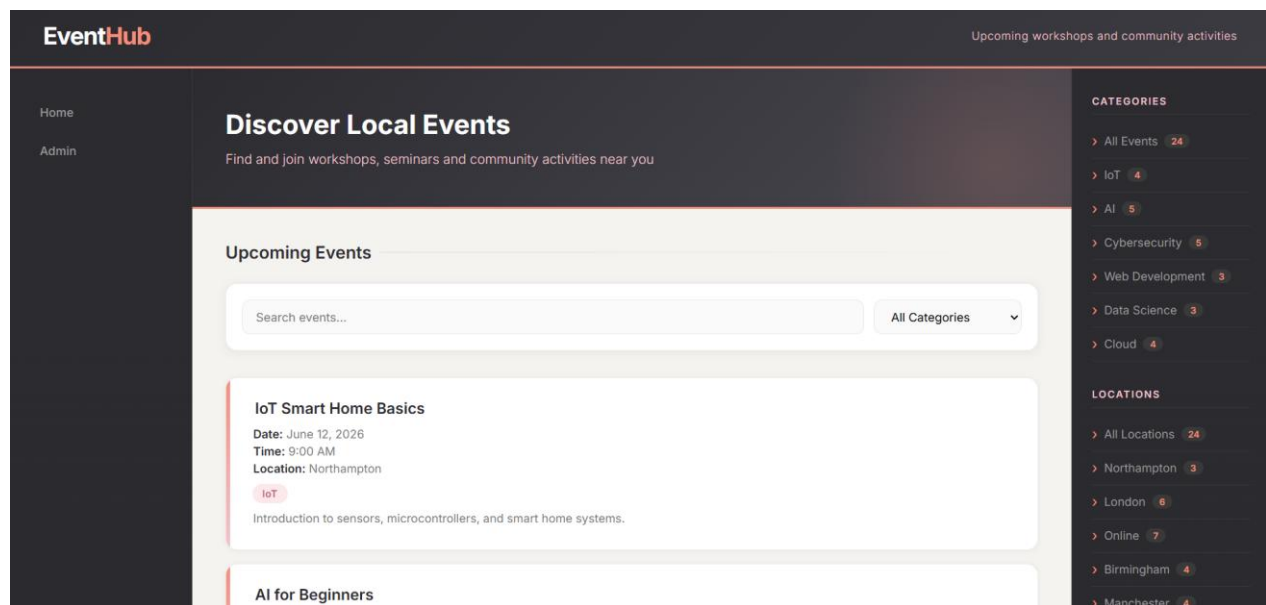


Figure 8: Public Events Listing Page

5.6 Search and Filter Functionality

The search and filter feature is the most technically interesting part of the application. It allows users to find events instantly without the page reloading.

When the user types in the search box, changes the category dropdown, or clicks a category or location in the sidebar, JavaScript captures the input and sends an AJAX GET request to `get_events.php` with the search term, category and location as URL parameters.

`get_events.php` receives these parameters and builds a dynamic SQL query using prepared statements. The base query selects all events, and WHERE clauses are added depending on which filters are active. The results are fetched from MySQL, encoded as a JSON array and returned to the browser.

JavaScript receives the JSON response and loops through the events, building new HTML event cards and replacing the existing content on the page. If no events match, a no results message is displayed.

Search and Filter:

```
CLIENT SIDE (JavaScript):
    ON search input OR category change OR location click
    GET search value, category value, location value
    SEND AJAX GET request to get_events.php
    with parameters: search, category, location

SERVER SIDE (PHP - get_events.php):
    GET search, category, location from GET parameters
    BUILD query: SELECT * FROM events WHERE 1=1
    IF search not empty THEN ADD: AND title LIKE '%search%'
    IF category not empty THEN ADD: AND category = category
    IF location not empty THEN ADD: AND location = location
    ADD: ORDER BY date ASC
    EXECUTE prepared statement
    ENCODE results as JSON
    RETURN JSON response

CLIENT SIDE (JavaScript - on response):
    PARSE JSON response
    IF results empty THEN show "No events found"
    ELSE
        LOOP through events
        BUILD HTML card for each event
        UPDATE events container on page
```

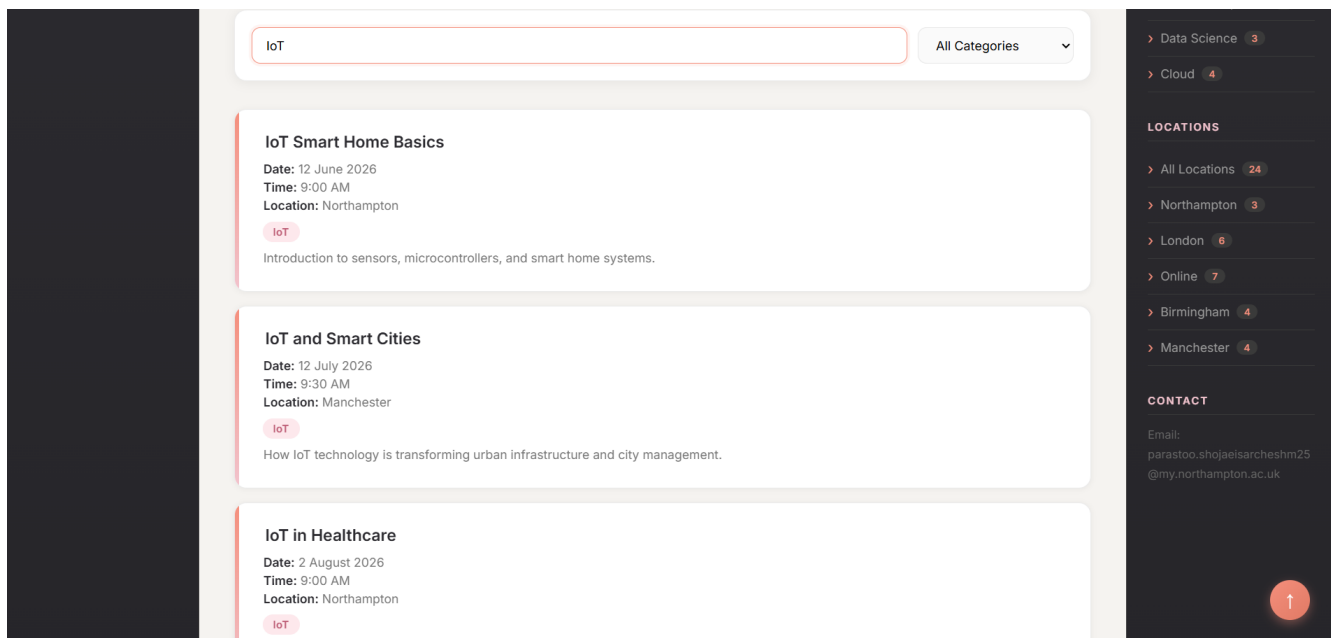


Figure 9: AJAX Search Filter in action

6 Use of Required Technologies

This section explains how each of the required technologies was used in the EventHub application.

6.1 HTML

HTML5 was used to structure all pages in the application. I used semantic elements throughout to make the code more readable and meaningful. The public page uses a header element for the site title, a main element for the events listing, an aside element for the category and location sidebar, and a footer element at the bottom.

The admin panel uses the same grid structure with additional form elements for adding and editing events. Each form uses label elements linked to their inputs with matching id and for attributes, which improves accessibility.

Event cards on the public page are built as div elements containing headings, paragraphs and span elements for the category badges. The search section uses an input element for the keyword search and a select element for the category dropdown.

6.2 CSS

[3], [5] CSS3 was used for all styling in the application. The layout is built using CSS Grid with the following template areas: header, nav, main, aside and footer. This makes the layout easy to understand and maintain.

The colour palette used throughout the site is based on five colours.

Event cards have a left border accent, a subtle box shadow, and a hover effect that lifts the card slightly using `translateY`. Category badges are colour-coded each of the six categories has its own background and text colour.

Responsive design was implemented using a CSS media query at 768px. Below this breakpoint, the grid switches from a three-column layout to a single-column layout so the site works on mobile devices.

The admin dashboard uses a grid of stat boxes to display event counts per category. These boxes have a coloured top border and a hover animation.

6.3 JavaScript

[4] JavaScript was used for three main purposes in the application.

The first is client side form validation. When the admin submits the add event form, JavaScript checks that all fields are filled in and that the selected date is not in the past. If any check fails, an error message is displayed and the form is not submitted.

The second is sending AJAX requests. JavaScript listens for input events on the search box, change events on the category dropdown, and click events on the sidebar category and location items. When any of these clicked, a fetch request is sent to the relevant PHP script with the filter parameters in the URL.

The third is updating the DOM dynamically. When the AJAX response returns, JavaScript parses the JSON and builds new HTML event cards, replacing the existing content in the events container without reloading the page. If no results are found, a no results message is displayed instead.

6.4 JSON

JSON is used as the data format for all AJAX responses in the application. When the public page sends a search or filter request, `get_events.php` queries MySQL and returns the results as a JSON array. Each object in the array represents one event and contains all the fields needed to build an event card title, date, time, location, category and description.

The admin panel also uses JSON responses. When an event is added or deleted, the PHP script returns a JSON object with a success field set to true or false and a message field explaining the result. JavaScript reads this response and shows either a success message or an error message to the admin.

6.5 AJAX

[4] AJAX is used throughout the application to update content without reloading the page. On the public site, the search box, category dropdown and sidebar filters all use AJAX to fetch filtered results from the server. This gives the user instant feedback as they type or click, rather than waiting for a full page reload.

On the admin side, the add event and delete event features both use AJAX. When an event is added, the form data is sent via a fetch POST request. When an event is deleted, a confirmation prompt appears and then a fetch POST request removes the event. In both cases, the page updates without a full reload.

All AJAX requests include error handling. If the server cannot be reached or returns an unexpected response, an error message is displayed to the user using JavaScript.

6.6 PHP

[7] PHP was used for all server-side processing in the application. The main responsibilities are:

- **Database connection:** db.php creates a PDO connection to MySQL using the server hostname, database name, username and password. This file is included by all other PHP scripts that need database access.
- **Session management:** login.php creates a PHP session when the admin logs in successfully. logout.php destroys the session. All admin pages check for a valid session at the top of the file.
- **CRUD operations:** add_event.php, edit_event.php and delete_event.php handle the three event management operations. Each file validates the input, runs a prepared statement and returns a response.
- **Search and filter:** get_events.php builds a dynamic SQL query based on the search, category and location parameters received from the AJAX request and returns the results as JSON.
- **Output sanitization:** all values retrieved from the database are passed through htmlspecialchars before being embedded in HTML to prevent cross-site scripting.

6.7 MySQL

MySQL 5.7 was used as the database for the application. It stores all event data and the admin credentials. The database is named Internet_programming and contains two tables — events and admins.

PHP connects to MySQL using PDO rather than the older mysqli extension. PDO supports prepared statements which protect against SQL injection by separating the SQL query from the data values.

The events table is queried in several ways — retrieving all events ordered by date for the public listing, filtering by keyword using LIKE, filtering by category or location using equality checks, retrieving a single event by ID for editing, and deleting a single event by ID.

The init.sql file creates both tables and inserts 24 sample events automatically when Docker starts, so the database is ready to use without any manual setup.

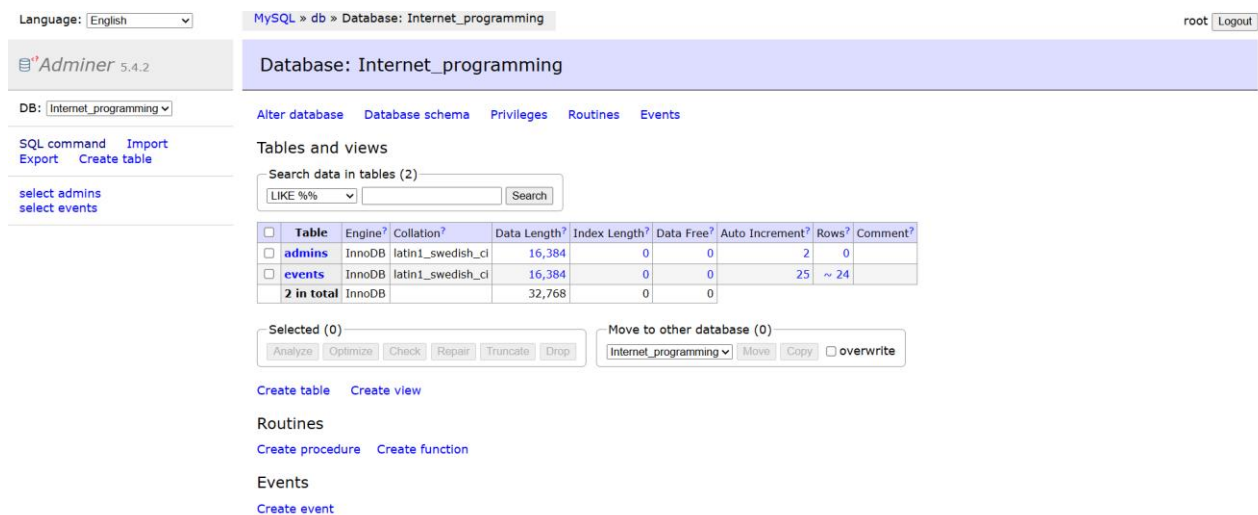


Figure 10: Internet_programming database

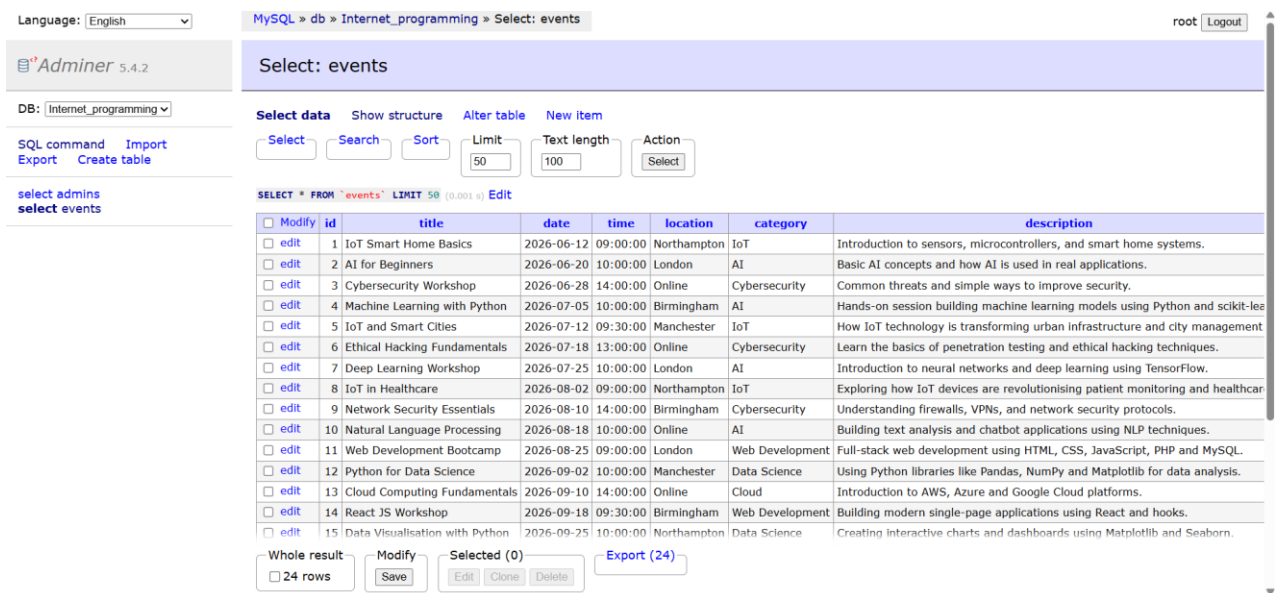


Figure 11: events table in MySQL

7 Security and Validation

Security was an important consideration throughout the development of EventHub. This section explains the measures I put in place to protect the application and its data.

7.1 Input Validation

Input validation happens on two levels, the client side using JavaScript and the server side using PHP.

On the client side, JavaScript checks the add event form before it is submitted. It verifies that the title, date, location and description fields are not empty, and that the selected date is not in the past. If any check fails, an error message is displayed and the AJAX request is not sent.

On the server side, PHP performs the same checks again independently. This is important because client-side validation can be bypassed by disabling JavaScript or sending a request directly to the PHP script. Server-side validation ensures that invalid data can never reach the database regardless of how the request was made.

Validation:

```
CLIENT SIDE:
    IF title is empty THEN show error, STOP
    IF date is empty THEN show error, STOP
    IF date is in the past THEN show error, STOP
    IF location is empty THEN show error, STOP
    IF description is empty THEN show error, STOP
    ELSE send AJAX request

SERVER SIDE:
    IF title OR date OR location OR category OR description is
empty
        RETURN JSON error response
        EXIT
    ELSE continue to database operation
```

7.2 Data Sanitisation

[6], [12] All data retrieved from the database is passed through PHP's htmlspecialchars function before it is embedded into HTML. This converts special characters such as angle brackets and ampersands into their HTML entities, which prevents cross-site scripting attacks where a malicious user might try to inject JavaScript code into an event title or description.

Output Sanitisation:

```
WHEN displaying any value from the database:
    OUTPUT htmlspecialchars(value)

Example:
<script>alert('xss')</script>
BECOMES:
&ltamplt;script&gt;alert(&#039;xss&#039;)&lt;/script&gt;
DISPLAYS as plain text, not executable code
```

7.3 SQL Injection Prevention

[7] , [11] , [13] All database queries in the application use PDO prepared statements. This means the SQL query structure is defined separately from the data values. The database driver handles the escaping of values automatically, so a malicious input such as a quote or SQL keyword cannot break out of the data context and modify the query structure.

Prepared Statement:

```
INSTEAD OF:
    query = "SELECT * FROM events WHERE title LIKE '%" + search +
    "%'"
    (dangerous - SQL injection possible)

USE:
    query = "SELECT * FROM events WHERE title LIKE :search"
    params = { search: "%" + search + "%" }
    PREPARE query
    EXECUTE with params
```

7.4 Admin Protection

[8], [9] The admin panel is protected using PHP sessions. When an admin logs in successfully, a session variable is set. Every admin page checks for this variable at the very top of the file before any other code runs. If the session variable is not set, the user is immediately redirected to the login page.

Passwords are never stored in plain text. When the admin account is created in init.sql, the password is stored as a bcrypt hash generated using PHP's password_hash function. When the admin logs in, PHP uses password_verify to compare the entered password against the stored hash. bcrypt is a slow hashing algorithm designed specifically for passwords it is computationally expensive to crack even if an attacker gains access to the database.

Session Protection:

```
AT TOP OF EVERY ADMIN PAGE:
    IF session variable 'admin' is NOT set THEN
        REDIRECT to login page
        EXIT
    END IF
    CONTINUE loading page
```

Password Hashing:

```
STORING password (done once in init.sql):
    hash = password_hash("admin123", BCRYPT)
    STORE hash in database

CHECKING password on login:
    GET hash FROM database WHERE username matches
```

```
IF password_verify(entered_password, stored_hash) THEN
    LOGIN successful
ELSE
    SHOW error message
```

8 Code Quality and Maintainability

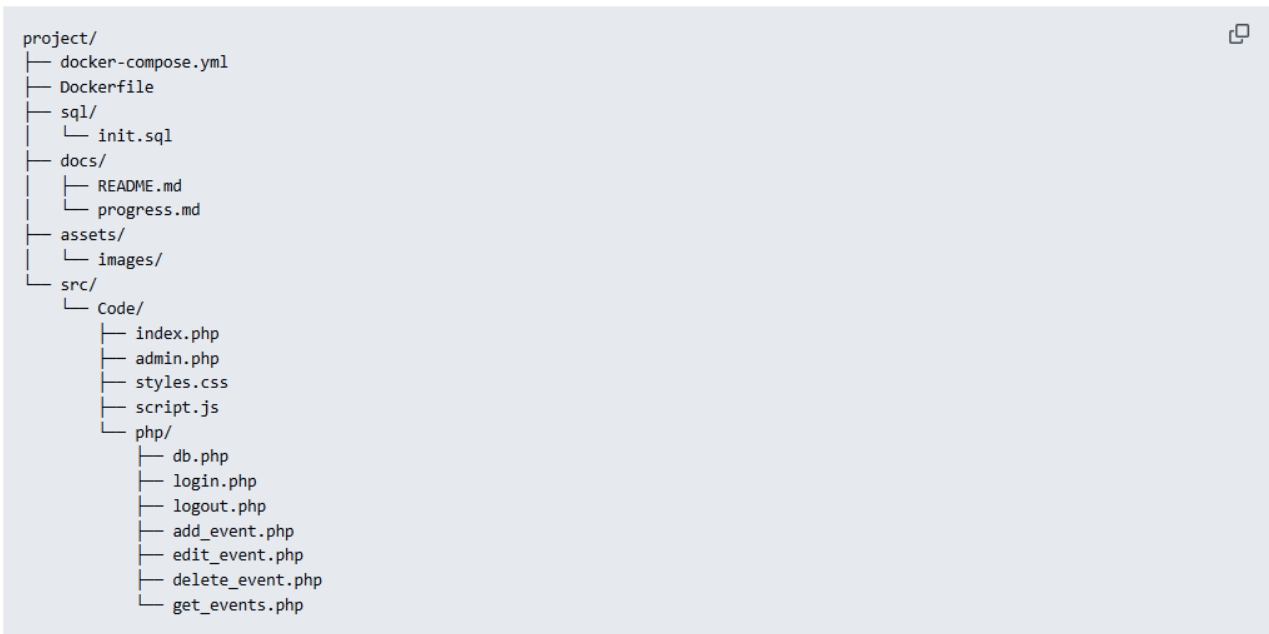
8.1 File Organisation

The project files are organised into a logical folder structure that separates different types of code. HTML and PHP page files sit in the src/Code directory. All PHP backend scripts are grouped together in a php subfolder. The CSS and JavaScript files sit alongside the main page files. The database schema is in a separate sql folder. Documentation is kept in a docs folder.

This separation means that a developer picking up the project can find what they are looking for quickly without having to search through a single large file.

README.md:

File Structure



```
project/
├── docker-compose.yml
├── Dockerfile
├── sql/
│   └── init.sql
├── docs/
│   ├── README.md
│   └── progress.md
├── assets/
│   └── images/
└── src/
    ├── Code/
    │   ├── index.php
    │   ├── admin.php
    │   ├── styles.css
    │   └── script.js
    └── php/
        ├── db.php
        ├── login.php
        ├── logout.php
        ├── add_event.php
        ├── edit_event.php
        ├── delete_event.php
        └── get_events.php
```

8.2 Reusable Code

The database connection is written once in db.php and included by every other PHP file that needs it using require. This follows the DRY principle so if the database credentials ever need to change, they only need to be updated in one place.

The CSS stylesheet is a single external file linked by all pages. This means the design is consistent across the public site and the admin panel, and any styling change only needs to be made once.

The JavaScript file is also a single external file. Common functions such as `showSuccess` and `showError` are defined once and called from multiple places throughout the script rather than being written out repeatedly.

8.3 Naming and Comments

I tried to use clear and descriptive names for variables and functions throughout the code.

Every PHP and JavaScript file includes comments explaining what each section does. These are written in plain English so that another developer could understand the code without needing to read through every line in detail.

For example:

```
// Send AJAX request to get filtered events
```

```
// Check that date is not in the past
```

```
// Save the event to the database using prepared statement
```

The comments are kept short and relevant. I avoided writing comments that simply repeat what the code already says, and instead focused on explaining why something is done or what the expected input and output are.

8.4 Semantic HTML

The HTML pages use semantic elements throughout rather than relying entirely on generic `div` elements. The page structure uses `header`, `nav`, `main`, `aside` and `footer` elements which describe the purpose of each section. Form elements use `label` elements properly linked to their inputs. Event cards use heading elements at the appropriate level.

This makes the HTML easier to read and also improves accessibility for screen readers and other assistive technologies.

8.5 GitHub Commit History

Version control was maintained throughout the project using GitHub Classroom. Commits were made regularly from February through to May 2026, showing that the project was developed gradually over time rather than being uploaded all at once.

The early commits in February and March show the initial HTML structure, CSS layout and JavaScript functionality being added. The April commits show the Docker setup and PHP backend being built. The May commits cover the AJAX search and filter features, UI improvements, category badges, location filtering, and finally the PHPUnit tests and documentation.

The full commit history is available at:

<https://github.com/D12AOS/csym019-internet-programming-assignment-prstshojaei>

9 Testing and Evaluation

9.1 Testing Strategy

I tested the application manually throughout development by running it in the browser and checking each feature worked as expected. I tested in both Chrome and Firefox to check cross-browser compatibility. I also wrote automated PHPUnit tests to verify the core application logic.

The main areas I tested were: admin login and session management, adding, editing and deleting events, public event listing, AJAX search and filter functionality, form validation with valid and invalid inputs, edge cases such as empty search results and missing required fields, and PHPUnit white-box testing of core functions.

9.2 Test Cases

The table below shows the manual test cases carried out during development:

ID	Feature	Input	Expected Result	Actual Result	Pass/Fail
T1	Admin login	Valid username and password	Session created, redirected to dashboard	Redirected successfully	Pass
T2	Admin login	Wrong password	Error message shown	Error message displayed	Pass
T3	Add event	All fields filled correctly	Event saved and appears on public site	Event appeared in list immediately	Pass
T4	Add event	Empty title field	Error message, form not submitted	Validation error displayed	Pass
T5	Add event	Past date selected	Error message shown	Client-side error displayed	Pass
T6	Edit event	Update event title	New title saved to database	Title updated correctly	Pass
T7	Delete event	Click delete and confirm	Event removed from database	Event disappeared from list	Pass
T8	Delete event	Click delete and cancel	Event not deleted	Event remained in list	Pass
T9	Public listing	Open homepage	All events loaded from database	24 events displayed correctly	Pass
T10	Search	Type 'IoT' in search box	Only IoT events shown without reload	Correct events displayed instantly	Pass
T11	Search	Random text with no match	No results message shown	No events found message displayed	Pass
T12	Category filter	Select AI from dropdown	Only AI events shown	Correct events displayed	Pass
T13	Location filter	Click London in sidebar	Only London events shown	Correct events filtered	Pass

T14	Category sidebar	Click Cybersecurity	Only Cybersecurity events shown	Correct events filtered	Pass
T15	Browser test	Open in Firefox	Site displays correctly	No layout issues found	Pass
T16	Responsive	Resize to mobile width	Layout switches to single column	Responsive layout worked	Pass

9.3 PHPUnit Test Results

I wrote 12 automated PHPUnit tests using PHPUnit 10 running inside the Docker container. All 12 tests passed with 18 assertions.

Test	What it checks	Result
testValidEventTitle	A non-empty title passes validation	Pass
testEmptyTitleFails	An empty title fails validation	Pass
testValidDateFormat	A valid date string is recognised correctly	Pass
testInvalidDateFormat	An invalid date string returns false	Pass
testValidCategory	A valid category is in the allowed list	Pass
testInvalidCategory	An invalid category is not in the allowed list	Pass
testDescriptionMinLength	A description meets the minimum length	Pass
testPasswordHashIsValid	A bcrypt hash verifies correctly	Pass
testWrongPasswordFails	A wrong password fails verification	Pass
testHtmlSpecialCharsRemovesXSS	XSS characters are escaped correctly	Pass
testEventDateIsInFuture	A future date is correctly identified	Pass
testValidTimeFormat	A time string has the correct format	Pass

PHPUnit terminal output:

```

Runtime:      PHP 8.1.34

.....                                     12 / 12 (100%)

Time: 00:00.212, Memory: 8.00 MB

OK (12 tests, 18 assertions)

```

Figure 12: PHPUnit terminal

9.4 Evaluation Against Requirements

The table below shows how each requirement from the assignment brief was addressed:

Requirement	Completed	Evidence
Admin can log in securely	Yes	Section 5.1, Figure 5
Admin can add events	Yes	Section 5.2, Figure 6
Admin can edit events	Yes	Section 5.3, Figure 7
Admin can delete events	Yes	Section 5.4
Public users can view all events	Yes	Section 5.5, Figure 8
Search by keyword works via AJAX	Yes	Section 5.6, Figure 9
Filter by category works via AJAX	Yes	Section 5.6
Filter by location works via AJAX	Yes	Section 5.6
PHP returns JSON responses	Yes	Section 6.4
MySQL database used for storage	Yes	Section 4
PDO prepared statements used	Yes	Section 7.3
Form validation on client and server	Yes	Section 7.1
Responsive design included	Yes	Section 6.2
PHPUnit tests written and passing	Yes	Section 9.3
GitHub commit history maintained	Yes	Section 8.5

10 Challenges and Reflections

10.1 Docker Configuration

[2] The most significant challenge encountered during development was configuring the Docker environment. The default PHP 7.2 Apache image does not include the PDO MySQL driver, which is required to connect to the database. A custom Dockerfile was written to install this driver using the `docker-php-ext-install` command.

A further issue arose when attempting to install PHPUnit. The Debian Buster package repositories used by PHP 7.2 were no longer available, which prevented `apt-get` from downloading packages. This was resolved by upgrading the base image to PHP 8.1, which uses a more recent version of Debian with functioning repositories. This change also allowed PHPUnit 10 to be installed, as it requires PHP 8 or higher.

A port conflict on port 3306 also prevented the containers from starting initially. This was resolved by removing the MySQL port mapping from `docker-compose.yml`, as the PHP container is able to reach the database through Docker's internal network without requiring an exposed port.

10.2 AJAX Implementation

Implementing the AJAX search and filter functionality presented several challenges. The initial requests were returning 404 errors because the file paths in JavaScript were relative rather than absolute. This was resolved by adding a leading slash to make the paths absolute from the web root.

A further issue was that JSON responses were being received correctly but the page was not updating. This was caused by JavaScript attempting to update a DOM element that did not exist on a particular page. A conditional check was added before the DOM update to resolve this.

Making the location and category filters work simultaneously required storing the current location selection as a JavaScript variable so that it could be included in every AJAX request regardless of which filter triggered it.

10.3 Form Validation

Initially only server-side validation was implemented. This meant the user had to wait for a round trip to the server before seeing an error message. Client-side JavaScript validation was added to provide immediate feedback. A check for past dates was also added because the browser date picker does not prevent the selection of past dates by default.

10.4 Design Decisions

The decision was made to keep the database structure simple with two tables rather than creating a separate categories table. For an application of this scale, storing the category as a text field in the events table is sufficient and keeps the queries straightforward.

The Fetch API was chosen over XMLHttpRequest for AJAX requests due to its cleaner syntax. PDO was chosen over mysqli for database access because it supports prepared statements more effectively and is considered better practice for modern PHP development.

Docker was used for the development environment to ensure the application can be set up consistently on any machine without requiring manual installation of PHP or MySQL.

10.5 Limitations and Future Improvements

Given more time, the following improvements would be made to the application:

- Pagination on the public events listing page as the number of events grows
- A date range filter to allow users to find events within a specific time period
- The ability for public users to register and sign up for events
- An image upload feature so that each event can have an associated photograph

11 Innovation and Additional Features

11.1 Location Filter in the Sidebar

In addition to the category filter required by the brief, a location filter was implemented in the sidebar. Each location is listed with a count showing how many events are held there. Clicking a location instantly filters the event list via AJAX to show only events in that location. This works alongside the category filter and keyword search, and an All Locations option resets the filter.

This feature improves the user experience by giving visitors an additional way to find events relevant to them, particularly useful for a community organisation that hosts events across multiple cities.

11.2 Event Counts in the Sidebar

Both the category list and the location list in the sidebar display a count of how many events are in each group. These counts are calculated in PHP by querying the database and are displayed as small badges next to each item. This gives users an at-a-glance overview of where the events are concentrated before applying any filter.

11.3 Coming Soon Badge

Events taking place within the next seven days automatically display a Coming Soon badge. This is calculated in PHP by comparing the event date against today's date using the DateTime class. The badge uses a CSS pulse animation to draw attention to upcoming events. This feature helps public users identify time-sensitive events quickly without having to check dates manually.

11.4 Admin Dashboard with Statistics

The admin panel includes a dashboard at the top of the page showing the total number of events and a breakdown of event counts across all six categories IoT, AI, Cybersecurity, Web Development, Data Science, and Cloud. These statistics are calculated dynamically from the database each time the admin panel loads, so they always reflect the current state of the data.

11.5 Event Time Field

A time field was added to the events table and included in both the add event and edit event forms. Event time is displayed on each event card on the public site alongside the date. This provides more useful information to visitors and makes the application more realistic as a community events portal.

11.6 Hero Section

A hero section was added to the top of the public page with a gradient background and a welcome message. This improves the visual presentation of the site and gives it a more professional appearance compared to a plain events list.

11.7 Human-Readable Date Format

Rather than displaying dates in the raw database format of 2026-06-12, PHP's date function is used to format dates as June 12, 2026. This makes the public listing page much easier to read and gives the application a more polished feel.

11.8 Back to Top Button

A back to top button appears in the bottom right corner of the page when the user scrolls more than 300 pixels down. This is particularly useful on the public events listing page which displays 24 events and requires significant scrolling. The button is implemented in JavaScript and styled to match the site's colour scheme.

11.9 PHPUnit Automated Testing

[10] PHPUnit automated tests were written to verify the core application logic. Twelve tests were implemented covering input validation, date format checking, category validation, password hashing and verification, XSS protection, and time format validation. All twelve tests pass with eighteen assertions. Writing these tests required upgrading the PHP image to PHP 8.1 and installing PHPUnit 10 via Composer inside the Docker container.

11.10 Colour-Coded Category Badges

Each of the six event categories has a distinct colour-coded badge displayed on every event card. The badge colours are consistent across the public site and the admin panel. This helps users identify the category of an event at a glance without having to read the category text.

11.11 Responsive Design

The application uses CSS Grid with a media query at 768px to provide a responsive layout. Below this breakpoint the three-column grid collapses to a single column, making the site usable on mobile devices. The search section and event cards also adapt to smaller screen widths.

12 Deployment and Setup Instructions

12.1 Required Software

To run the EventHub application, the following software must be installed:

- Docker Desktop (available at <https://www.docker.com/products/docker-desktop>)
- A modern web browser such as Chrome or Firefox
- Git (to clone the repository)

No manual installation of PHP or MySQL is required as these are handled by Docker.

12.2 Installation Steps

1. Clone the repository from GitHub Classroom:

```
git clone https://github.com/D12AOS/csym019-internet-programming-assignment-prstshojaei.git
```

2. Navigate into the project folder:

```
cd csym019-internet-programming-assignment-prstshojaei
```

3. Start Docker Desktop and wait for the engine to show as running.

4. Start the Docker containers:

```
docker compose up
```

5. Wait for the following message to appear in the terminal:

```
mysqld: ready for connections
```

6. Open the public site in your browser:

```
http://localhost
```

7. To access Adminer and view the database:

```
http://localhost:8080
```

12.3 Admin Login Details

Setting	Value
Admin URL	http://localhost/php/login.php
Username	admin
Password	admin123

12.4 Database Credentials

These credentials are used if accessing the database directly through Adminer at <http://localhost:8080>.

Setting	Value
Server	db
Username	root
Password	csym019
Database	Internet_programming

12.5 Stopping the Application

To stop the application, press Ctrl+C in the terminal, or run:

```
docker compose down
```

To stop the application and remove all data including the database:

```
docker compose down -v
```

13 Conclusion

EventHub is a fully functional community events management web application built using HTML, CSS, JavaScript, PHP, MySQL, AJAX and JSON. The application meets all of the core requirements set out in the assignment brief and includes a number of additional features that improve the user experience and demonstrate a deeper engagement with the technologies involved.

The public-facing site allows visitors to browse all upcoming events, search by keyword, and filter by category and location, all without the page reloading. The admin panel provides a secure interface for staff to add, edit and delete events, with form validation on both the client and server side to ensure data integrity.

All database queries use PDO prepared statements to prevent SQL injection, and all output is escaped using htmlspecialchars to prevent cross-site scripting. The admin panel is protected by PHP sessions and passwords are stored as bcrypt hashes. These security measures ensure that the application follows industry best practices for web security.

The code is organised into separate files for HTML, CSS, JavaScript and PHP, with a single reusable database connection file included wherever it is needed. Comments are included throughout the code to explain the purpose of each section. The project uses Docker to provide a consistent development environment that can be set up on any machine with a single command.

Automated PHPUnit tests were written to verify the core application logic, with all twelve tests passing. The GitHub Classroom repository contains a commit history spanning from February to May 2026, demonstrating that the project was developed incrementally over time.

The main limitation of the current implementation is the lack of pagination on the public events listing page. Future improvements would include pagination, a date range filter, public user registration for events, and image upload functionality for event photos.

Overall, the development of EventHub provided valuable experience in building a full-stack web application that integrates multiple technologies in a coherent and maintainable way.

14 AI Usage Statement

Generative AI was used in an assistive role during this project, primarily for debugging code errors and identifying issues during development. All design decisions, implementation and written content are my own work.

15 References

- [1] Connolly, T. and Begg, C. (2014) Database Systems: A Practical Approach to Design, Implementation and Management. 6th edn. Harlow: Pearson Education.
- [2] Docker Inc. (2024) Docker Documentation. Available at: <https://docs.docker.com>
- [3] Google Fonts (2024) Inter Font Family. Available at: <https://fonts.google.com/specimen/Inter>
- [4] Mozilla Developer Network (2024) Fetch API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [5] Mozilla Developer Network (2024) CSS Grid Layout. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout
- [6] Mozilla Developer Network (2024) HTMLspecialchars — Output Escaping. Available at: https://developer.mozilla.org/en-US/docs/Glossary/Cross-site_scripting
- [7] PHP Group (2024) PHP Manual — PDO. Available at: <https://www.php.net/manual/en/book.pdo.php>
- [8] PHP Group (2024) PHP Manual — password_hash. Available at: <https://www.php.net/manual/en/function.password-hash.php>
- [9] PHP Group (2024) PHP Manual — Sessions. Available at: <https://www.php.net/manual/en/book.session.php>
- [10] PHPUnit (2024) PHPUnit Documentation. Available at: <https://docs.phpunit.de>
- [11] OWASP Foundation (2024) SQL Injection Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [12] OWASP Foundation (2024) Cross Site Scripting Prevention Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- [13] W3Schools (2024) PHP MySQL Prepared Statements. Available at: https://www.w3schools.com/php/php_mysql_prepared_statements.asp

16 Appendices

16.1 Appendix A: Project File Structure

File Structure

```
project/
├── docker-compose.yml
├── Dockerfile
├── sql/
│   └── init.sql
├── docs/
│   ├── README.md
│   └── progress.md
├── assets/
│   └── images/
├── src/
│   ├── Code/
│   │   ├── index.php
│   │   ├── admin.php
│   │   ├── styles.css
│   │   ├── script.js
│   │   └── php/
│   │       ├── db.php
│   │       ├── login.php
│   │       ├── logout.php
│   │       ├── add_event.php
│   │       ├── edit_event.php
│   │       ├── delete_event.php
│   │       └── get_events.php
```

16.2 Appendix B: Database Sample Data

The init.sql file inserts 24 sample events across six categories on startup:

Category	Number of Events
IoT	4
AI	5
Cybersecurity	5
Web Development	3
Data Science	3
Cloud	4
Total	24

16.3 Appendix C: PHPUnit Test File Location

The PHPUnit test file is located at:

```
src/Code/tests/EventTest.php
```

To run the tests, execute the following command after starting Docker:

```
docker exec -it csym019-internet-programming-assignment-
prstshojaei-php-1 phpunit /var/www/html/tests/EventTest.php
```